

Competitive Programming

Section 53 Editorial: Advanced Combinatorics

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Catalan Queries	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Derangement Queries	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Necklace Orbits	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	D. Unlabeled Teams	10
5.1	Restatement and model	10
5.2	Recurrence and trace	10
5.3	Algorithm	10
5.4	Correctness proof	11
5.5	Complexity, implementation notes, and tests	11
5.6	Full C++ solution	11
5.7	Full Python solution	12
6	What to carry forward	13
7	Complete upsolve notes	14
7.1	A. Catalan Queries	14
7.2	B. Derangement Queries	14
7.3	C. Necklace Orbits	14

1 Editorial overview

1.1 Problem-specific proof map

A. Catalan Queries. The recurrence splits a balanced structure at its first matching close, producing two independent smaller Catalan objects. Precomputed factorials evaluate the equivalent binomial formula modulo the prime.

B. Derangement Queries. Condition on where element n goes: its displaced partner either returns to n or joins a derangement of the remaining elements, giving $D[n] = (n-1)(D[n-1] + D[n-2])$.

C. Necklace Orbits. Burnside averages the number of colorings fixed by each rotation. A rotation by r has $\gcd(n, r)$ cycles, hence fixes $k^{\gcd(n, r)}$ colorings.

D. Unlabeled Teams. For the final labeled person, either create one new singleton team or join one of the existing teams. This produces the Stirling recurrence $S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$.

1.2 Detailed solution guide

A. Catalan Numbers. The number of balanced parenthesis strings with n pairs is $C_n = \text{binom}(2n, n) / (n+1)$. The binomial chooses the positions of the opening brackets, and the division removes the invalid paths counted by the reflection argument. With a prime modulus, compute factorials and inverse factorials, then multiply by the modular inverse of $n+1$.

B. Derangements. Let $D[n]$ be permutations of n elements with no fixed points. Look at where element 1 goes, say to j . Either j maps back to 1, leaving $D[n-2]$ choices, or j does not map back, which can be converted to a derangement-like arrangement on $n-1$ elements. This gives $D[n] = (n-1)(D[n-1] + D[n-2])$.

C. Burnside Rotations. Burnside's lemma says the number of necklaces is the average number of colorings fixed by each rotation. A rotation by r positions has $\gcd(n, r)$ cycles, and each cycle must have one color, so it fixes $k^{\gcd(n, r)}$ colorings. Sum this over all rotations and divide by n .

Common pitfalls. Modular division means multiplying by an inverse. Burnside counts orbits, not representatives you construct manually. Derangement bases are $D[0]=1$, $D[1]=0$.

This section is about selecting the correct counting theorem before coding.

- A evaluates Catalan numbers with factorials and modular inverses: $C(2n, n) / (n+1)$.
- B precomputes derangements using $D[n] = (n-1) * (D[n-1] + D[n-2])$.
- C applies Burnside's lemma to rotations of a necklace: average $k^{\gcd(n, r)}$ over all rotations r .

The correctness arguments identify the counted structure and then cite the corresponding recurrence or orbit-counting lemma with its assumptions.

2 A. Catalan Queries

2.1 Restatement

Print Catalan numbers modulo $1e9+7$.

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; long long pw(long long a,long long e){l
```

2.6 Full Python solution

```
import sys
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    qs=data[1:1+data[0]]; N=max(qs+[0])*2; fact=[1]*(N+1)
    for i in range(1,N+1): fact[i]=fact[i-1]*i%MOD
    inv=[1]*(N+1); inv[N]=pow(fact[N],MOD-2,MOD)
```

```

for i in range(N,0,-1): inv[i-1]=inv[i]*i%MOD
def C(n,k): return fact[n]*inv[k]%MOD*inv[n-k]%MOD if 0<=k<=n else 0
print("\n".join(str(C(2*n,n)*pow(n+1,MOD-2,MOD)%MOD) for n in qs))
if __name__=="__main__": main()

```

3 B. Derangement Queries

3.1 Restatement

Print the number of derangements of n modulo $1e9+7$.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; int main(){ios::sync_with_stdio(false);
```

3.6 Full Python solution

```
import sys
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    qs=data[1:1+data[0]]; N=max(qs+[1]); d=[0]*(N+1); d[0]=1
    if N>=1: d[1]=0
    for i in range(2,N+1): d[i]=(i-1)*(d[i-1]+d[i-2])%MOD
```

```
    print("\n".join(str(d[n]) for n in qs))  
if __name__=="__main__": main()
```

4 C. Necklace Orbits

4.1 Restatement

Count color necklaces of length n with k colors up to rotation.

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; long long pw(long long a,long long e){l
```

4.6 Full Python solution

```
import sys,math
MOD=1_000_000_007
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    t=data[0]; idx=1; out=[]
    for _ in range(t):
        n,k=data[idx],data[idx+1]; idx+=2; s=0
```

```
    for r in range(n): s=(s+pow(k,math.gcd(n,r),MOD))%MOD
    out.append(str(s*pow(n,MOD-2,MOD)%MOD))
    print("\n".join(out))
if __name__=="__main__": main()
```

5 D. Unlabeled Teams

5.1 Restatement and model

Split n distinct people into exactly k nonempty teams. Teams are unlabeled: swapping two whole teams does not make a new partition. Count the partitions modulo $1,000,000,007$, with $1 \leq k \leq n \leq 2000$.

This is not k^n , because that assigns labels to teams and permits empty teams. It is also not simply $k!$ times a value after the fact unless every labeled assignment uses all teams. We build the count directly by the number of people and the number of nonempty unlabeled teams.

5.2 Recurrence and trace

Let $S(i, j)$ be the number of partitions of the first i people into exactly j nonempty teams. Focus on person i :

1. They form a singleton team. The first $i-1$ people must form $j-1$ teams, giving $S(i-1, j-1)$ choices.
2. They join an existing team. The first $i-1$ people form j teams, and there are j teams to join, giving $j \cdot S(i-1, j)$ choices.

Thus:

$$\begin{aligned} S(i, j) &= S(i-1, j-1) + j \cdot S(i-1, j) \\ S(0, 0) &= 1 \end{aligned}$$

For $S(4, 2)$:

$$\begin{aligned} S(1, 1) &= 1 \\ S(2, 1) &= 1, \quad S(2, 2) = 1 \\ S(3, 2) &= S(2, 1) + 2 \cdot S(2, 2) = 3 \\ S(4, 2) &= S(3, 1) + 2 \cdot S(3, 2) = 1 + 6 = 7 \end{aligned}$$

5.3 Algorithm

Use a one-dimensional array where $dp[j]$ holds the previous/current row for $S(_, j)$. Update j in decreasing order so $dp[j-1]$ still represents the previous row:

```

dp[0] = 1
for people from 1 to n:
    for teams from min(people,k) down to 1:
        dp[teams] = dp[teams-1] + teams * dp[teams] modulo MOD
    dp[0] = 0
print dp[k]

```

Setting `dp[0]` to zero after the first row matters: no positive number of people can be split into zero nonempty teams.

5.4 Correctness proof

Consider any partition of i people into j nonempty teams. Person i is either alone in their team or is not. In the first case, deleting them leaves one and only one partition of the first $i-1$ people into $j-1$ teams. In the second case, deleting them leaves a partition into j teams and records one of its j teams as the one they rejoin. These cases are disjoint and cover all partitions, proving the recurrence.

The descending update computes exactly that recurrence from the prior row, and the base $S(0,0)=1$ is the unique empty partition. By induction over `people`, after the final row $dp[k]=S(n,k)$, so the algorithm prints the required count. QED.

5.5 Complexity, implementation notes, and tests

The DP takes $O(nk)$ time and $O(k)$ memory. Multiply in 64-bit before taking the modulus in C++; Python integers are exact. Test $k=1$, $k=n$, small values such as $S(4,2)=7$, and a middle-sized k where no closed-form shortcut is available. A common bug is iterating teams upward, which overwrites the previous-row value needed for $S(i-1, j-1)$.

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007LL;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    if (!(cin >> n >> k)) return 0;
    vector<long long> dp(k + 1, 0);
    dp[0] = 1;
    for (int people = 1; people <= n; people++) {
        for (int teams = min(people, k); teams >= 1; teams--) {

```

```

        dp[teams] = (dp[teams - 1] + teams * dp[teams]) % MOD;
    }
    dp[0] = 0;
}
cout << dp[k] << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, k = data
    dp = [0] * (k + 1)
    dp[0] = 1
    for people in range(1, n + 1):
        for teams in range(min(people, k), 0, -1):
            dp[teams] = (dp[teams - 1] + teams * dp[teams]) % MOD
        dp[0] = 0
    print(dp[k])

if __name__ == "__main__":
    main()

```

6 What to carry forward

7 Complete upsolve notes

7.1 A. Catalan Queries

Precompute $C[0]=1$ and either the convolution $C[n]=\sum(C[i]*C[n-1-i])$ for the local bound or factorial formula under a legal prime modulus. The first pair decomposition is a bijection, proving the recurrence. State the actual precompute limit, use long long products, and test $n=0,1,2$. A parenthesis enumerator is a small oracle.

7.2 B. Derangement Queries

Use $D[0]=1, D[1]=0, D[n]=(n-1)(D[n-1]+D[n-2])$. Split on whether item one maps back to its partner or enters a longer cycle; the two cases are disjoint and complete. Precompute through maximum query in $O(\max N)$, test $0/1/2$, and reduce every product modulo MOD.

7.3 C. Necklace Orbits

Burnside averages fixed colorings: answer is $\text{inverse}(n) * \sum_{r=0}^{n-1} k^{\gcd(n,r)}$. A coloring is fixed by rotation r exactly when it is constant on $\gcd(n,r)$ cycles. $O(n \log n)$ per local query; test $n=1$, prime n , and composite n . Fermat inverse requires the stated prime modulus and n nonzero modulo it.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.

For D, tiny tests enumerate canonical restricted-growth representations of set partitions, independently of the recurrence. The scale case is a fixed $n=2000, k=1000$ instance with a precomputed expected residue.